

# **Sprint 6 — Observer Pattern**

## **Class 2**

### **Lesson 7 — Synchronous vs Asynchronous Observer**

#### **Learning Objective**

By the end of this lesson, we should understand:

- Synchronous Observer
- Asynchronous Observer
- Advantages and disadvantages of each
- Which one Spring Events and Kafka use

#### **Start With a Question**

Suppose a user registers.

We notify : Email, SMS, Audit, Analytics

Question:

Should User Registration wait for Email?

#### **Synchronous Observer**

User Register -> Email -> SMS -> Audit -> Analytics -> Return Response

Everything happens one by one.

## Example

```
publisher.publish(user);
```

```
System.out.println("Registration completed");
```

Output:

Save User

Send Email

Send SMS

Audit

Analytics

Registration completed

## Advantages

- Easy to debug

- Simple

- Immediate consistency

## Disadvantages

Slow.

If Email takes 5 seconds => Registration waits 5 seconds.

# Asynchronous Observer

Instead

Save User -> Return Response ->Background Processing

Email

SMS

Audit

Analytics

## Example

```
CompletableFuture.runAsync(() ->
    emailObserver.onUserRegistered(user));
```

Output:

Save User

Registration completed

...

Email

SMS

Analytics

## Advantages

-Fast response.

- Scalable.
- Observers run independently.

## Disadvantages

- More difficult to debug.
- Events may arrive later.
- Requires retry strategy.

## Getting Point

Spring Events: Default -> Synchronous

Kafka -> Asynchronous

## Lesson 8 — Observer vs Publish/Subscribe

We often think they are identical.

They are related, but not the same.

## Observer

Publisher knows the observers.

Publisher -> Observer A, Observer B, Observer C

Usually inside one application.

# Publish/Subscribe

Publisher -> Broker -> Subscribers

Publisher -> Kafka -> Subscriber A, Subscriber B, Subscriber C

Publisher never knows subscribers.

## Easy Rule

Observer : Same JVM

Publish/Subscribe: Different Applications

## Enterprise Example

Spring Event:

UserService -> EmailListener

It is Observer.

Kafka:

User Service -> Kafka -> Notification Service, Analytics Service, Discovery Service

It is Publish/Subscribe.

## Lesson 9 — Java Observer

Older Java versions had Observable, Observer

Example

```
Observable observable = new Observable();
```

Why don't we use it anymore?

Because it was deprecated.

Problems

- Uses inheritance instead of interfaces.
- Not flexible.
- Poor API design.
- Hard to extend.

Modern Java prefers interfaces and event publishers.

## Lesson 10 — Observer in Spring

### Event

```
public record UserRegisteredEvent(  
    User user  
) {  
}
```

### Publish Event

```
@Service  
@RequiredArgsConstructor  
public class UserService {  
  
    private final ApplicationEventPublisher publisher;
```

```
public void register(User user) {  
  
    System.out.println("Save User");  
  
    publisher.publishEvent(  
        new UserRegisteredEvent(user));  
  
}  
  
}
```

## Email Listener

```
@Component  
public class WelcomeEmailListener {  
  
    @EventListener  
    public void handle(UserRegisteredEvent event) {  
  
        System.out.println(  
            "Send Email to "  
            + event.user().getEmail());  
  
    }  
  
}
```

## SMS Listener

```
@Component  
public class SmsListener {  
  
    @EventListener
```

```
public void handle(UserRegisteredEvent event) {  
  
    System.out.println(  
        "Send SMS");  
  
}  
  
}
```

## Audit Listener

```
@Component  
public class AuditListener {  
  
    @EventListener  
    public void handle(UserRegisteredEvent event) {  
  
        System.out.println(  
            "Audit");  
  
    }  
  
}
```

## Getting Point

Notice: UserService knows only ApplicationEventPublisher

Not Email, SMS, Audit

This is Beautiful.

## Lesson 11 — Kafka Observer



Now connect to room platform.

Room Approved:

Room Service -> Kafka ->

- Discovery Service
- Notification Service
- Analytics Service
- Search Cache

Room Service knows none of them.

Property Submitted:

Property Service -> Kafka -> Moderation Service

Subscription Updated:

Subscription Service -> Kafka -> Discovery Service

This is Observer Pattern , distributed across microservices.

## **Lesson 12 — Common Mistakes**

### **Mistake 1**

Business transaction depends on observer. => Wrong

Save User -> Email fails -> Rollback User

Email shouldn't determine whether registration succeeds.

Use asynchronous processing for non-critical tasks.

## **Mistake 2**

One giant event

EverythingChangedEvent

Nobody knows what changed.

Instead

UserRegisteredEvent

RoomApprovedEvent

PaymentCompletedEvent

Small, specific events.

## **Mistake 3**

Publishing too many events

Every setter -> Publish event. => Bad.

Publish meaningful business events.

## **Mistake 4**

Observer modifying publisher. => Bad design.

Publisher publishes.

Observer reacts.

Keep responsibilities separate.

# Enterprise Example

Room Platform:

When owner submits a room

=> Room Service publishes RoomSubmittedEvent

Observers:

- Moderation Service
- Notification Service
- Analytics Service

When room approved => Moderation publishes RoomApprovedEvent

Observers:

- Discovery Service
- Search Ranking
- Notification

## Homework

Student Registration System

Student registered -> Issue Student ID -> Generate QR -> Send Welcome Email ->

Create LMS Account -> Notify Teacher

Implement

**Plain Java**

Publisher

Observers

**Spring**

ApplicationEventPublisher

EventListener

## Observer Pattern Summary

| Concept                      | Description                                                            |
|------------------------------|------------------------------------------------------------------------|
| <b>Publisher (Subject)</b>   | Produces an event when something meaningful happens                    |
| <b>Observer (Listener)</b>   | Reacts to the published event                                          |
| <b>Event</b>                 | Carries the business information observers need                        |
| <b>Synchronous Observer</b>  | Observers run immediately in the same execution flow                   |
| <b>Asynchronous Observer</b> | Observers run later, often in the background                           |
| <b>Spring Event</b>          | In-process Observer implementation                                     |
| <b>Kafka / RabbitMQ</b>      | Distributed Publish/Subscribe built on similar event-driven principles |

## Final Comparison of the Patterns Learned

| Pattern                | Key Question It Answers                                       |
|------------------------|---------------------------------------------------------------|
| <b>Strategy</b>        | Which algorithm or behavior should execute?                   |
| <b>Factory</b>         | Which object should be created?                               |
| <b>Builder</b>         | How should a complex object be constructed?                   |
| <b>Template Method</b> | Which steps of a workflow can subclasses customize?           |
| <b>Observer</b>        | How can one object notify many others without tight coupling? |